DistributedPanGA Objective O1: HySortK-Inspired Extraction

Final report

Harshaan Chugh
Cornell University
Ithaca, NY, USA

Abstract

DistributedPanGA distributes FastGA's all-vs-all pangenome alignment over MPI. Objective O1 of our proposal was to speed up its first stage, seed extraction, using ideas from the HySortK distributed k-mer counter. Measurement showed that extraction was dominated not by sorting or communication but by one `lseek` system call per extracted record. Packing each seed from the sequence window already in memory (O1a) made extraction 34–136× faster and the whole pipeline 1.7–3.7× faster on one node. Every run produced output identical to FastGA's. After O1a, extraction is about 2% of the run time. The remaining planned HySortK adaptations (threaded extraction, destination-aware staging and overlapped exchange) could not pay off and were dropped. Supermer compression does not apply to our seeds. We also report which HySortK ideas transfer to this pipeline and which do not.

1 Introduction

FastGA [5] aligns genomes on a single node. DistributedPanGA runs FastGA's self-alignment across MPI ranks in six stages: (1) *extract* 40-mer seeds chosen by closed (12,8)-syncmers, keeping each occurrence's contig, position and strand; (2) *sort* them globally by a distributed sort with a prefix-range exchange; (3) *merge* equal k-mers into seed pairs with FastGA's adaptive-seed merge; (4) *redistribute* seeds to the rank that owns each contig pair; (5) *align* each contig pair; and (6) *convert* the result to PAF.

HySortK [4] is a distributed sorting-based k-mer counter. It uses supermers, a task layer with balanced dispatch, hybrid MPI/OpenMP and batched non-blocking exchange. Objective O1 was to adapt its ideas to stages 1–2, while keeping every occurrence and alignments identical to FastGA's.

2 Verification method

Every claim below is checked against an *independent* reference: FastGA (revision `178afcb`) run on the same input. The pipeline's per-rank PAF files are concatenated and sorted without removing duplicates, then compared byte for byte with FastGA's PAF.

The main dataset is yeast7, which reproduces the historical 43,126 alignments. Additional phase verifiers check extraction (byte-identical to FastGA's index), the local and distributed sorts, and the LCP values.

Timings use one Perlmutter CPU node (2×AMD EPYC 7763) with 16 cores per rank, one thread per rank, and 3 interleaved repetitions. Reported spreads are sample standard deviations.

3 Diagnosis (O1-0)

Per-rank instrumentation on yeast7 (job 59679320) found:

- **Work was balanced.** Records per rank were within ±11% at 8 ranks.

Table 1: O1a on yeast7, same node, 3 interleaved repetitions (job 59691033). Kernel time is summed over ranks.

	1 rank		8 ranks	
	before	O1a	before	O1a
wall time (s)	113.80	**65.43**	39.11	**10.57**
extraction (s)	49.76	**1.48**	28.61	**0.21**
kernel time (s)	26.7	2.7	199.3	6.3
alignments vs. gold	match	match	match	match

- **The cost per record grew with the rank count**: 0.78, 0.95, 1.48 and 3.97 μs at 1, 2, 4 and 8 ranks.
- **User time scaled as $1/N$, but kernel time per rank stayed flat** at 19–30 s. Total kernel time rose from 26 s to 238 s for the same total work.
- **One `lseek` per record.** `strace` on the serial extractor counted 38,035 `lseek` calls for 38,023 records. Every seed re-read 40 bases through `Get_Contig_Piece`, whose `fseeko` issues a seek.
- **Output does not depend on pre-sort order.** Shuffling the received records before sorting left the per-rank PAFs byte-identical (yeast7, 1 and 4 ranks, 3 seeds). This is evidence for this input, not a proof.

4 O1a: pack from the scan window

The scanner already holds each contig window in memory. O1a packs each seed's forward and reverse-complement k-mer straight from that window.

Windows after the first re-read 40 bases of left context, so seeds near a window's start are covered. A forward k-mer that crosses a non-final window's right edge falls back to the old read. On these datasets every contig fits in one window, so the fallback never fired.

Seed selection, tie handling, coordinates, emission order and LCP handling are unchanged.

Table 1 shows extraction 34× faster at 1 rank and 136× faster at 8 ranks. The 1-to-8-rank speedup of the whole run rose from 2.9× to 6.2×.

Removing exactly the per-record seeks removed 97% of the 1-rank extraction time, which strongly supports the diagnosis. Which kernel path was contended at 8 ranks remains unattributed; it no longer affects performance.

All 20 O1a timing runs matched gold, as did all phase verifiers and an ASan/UBSan build of the multi-window extraction tests.

5 Dropped steps and why

Table 2 lists the other planned sub-steps. Each was evaluated against measurements taken after O1a.

Table 2: Planned O1 steps that were not built, with the measurement that decided each.

Step	Decision and evidence
O1b: threaded extraction	Dropped. After O1a, extraction is 0.21 of 10.57 s at 8 ranks (≈2%). Threads could save at most ≈1.5 s at 1 rank.
O1c: destination-aware staging (HySortK's per-thread × per-destination buffers)	Dropped. The speed gain is bounded by the histogram and packing passes, ≈0.45 s. When proposed it could not lower peak memory either, because the peak was in the alignment stage. Once later work freed those buffers, stage 2 became the 1-rank peak (2.96 GiB), so a memory-only variant is a possible future option.
O1d: batched, overlapped exchange	Dropped. The k-mer exchange takes ≤0.13 s per rank on one node, so there is nothing to hide. Exchanges above 2 GB per rank were later handled with bounded rounds instead.
Supermer compression	Not applicable. A seed's destination is its own first 5 bases, and consecutive selected seeds (≈2.5 bases apart) reach the same rank with probability $\approx 1/P$. The expected run is $P/(P-1)$ seeds, a net loss after headers. Routing by minimizer would split the 12-base prefix groups that the merge needs kept together.
Reusing HySortK code	Not done. The repository has no licence, so only ideas were reused.

6 What transfers from HySortK

- **Does not transfer:**
  - Supermers and heavy-hitter handling, which rely on k-mer-counting semantics.
  - Extraction-side staging and overlap, because our cost was a per-record seek.
  - HySortK's *static, size-based task dispatch*. We tested it on the alignment stage, which became the bottleneck after O1a. A task's cost there follows how much sequence it aligns, not its seed count: $R^2 = 0.19$ per task on primates4.chr6 [1]. Greedy dispatch by seed count was no better than the existing length-based assignment, and at 8 ranks it was worse.
- **Transfers:** the *task abstraction*, i.e. many independent units of work, more than there are ranks. It frames the follow-on scheduling work.

7 Follow-on work (outside O1)

Infrastructure (pull requests open or merged).

- An `ALNtoPAF` time limit, with a non-zero exit on failure.
- Per-stage instrumentation, plus freeing dead buffers early: peak memory −25 to −38%.
- Exchanges above 2 GB per rank, in bounded rounds. Rank outputs are byte-identical with many rounds, and the seed exchange's peak dropped from 2.5 GB to 40 MB.
- Scratch files can be placed off memory-backed `/tmp`.

Larger datasets. Inputs from the PGGB data [1, 3] all match FastGA gold:

- scerevisiae8: 50,046 alignments;
- primates4.chr6: 1,547,485 alignments, at 4–16 ranks;
- primates14.chr6: 4,577,556 alignments, at 16 ranks and at 32 ranks across 2 nodes.

Alignment scheduling (proposal O3).

- On primates14.chr6 the alignment stage is 96–97% of the run time. The slowest rank does 3.35× the average at 32 ranks.
- Self-alignment tasks are 49% of the work.
- Modelled from the measured task times, a dynamic task queue that runs self-alignments of long contigs first would cut the slowest rank's alignment time by 33–42% at 16–32 ranks.
- Beyond about 16 ranks, the single largest task (773 s) is a floor for any scheduler. Scaling further requires splitting self-alignment tasks while keeping FastGA-identical output. That is an open question; prior distributed aligners such as diBELLA [2] also leave work-aware alignment assignment open.

8 Conclusion

O1 is complete. One targeted change, found by measurement rather than by porting a technique, made extraction negligible: 1.5 s of 65 s at 1 rank and 0.2 s of 10.6 s at 8, with output identical to FastGA's.

The HySortK-inspired extensions planned for extraction were evaluated with data and dropped. Of HySortK's architecture, the task abstraction carries forward; size-based dispatch does not, because alignment cost depends on output rather than input.

References

[1] [n. d.]. Pangenomes of multiple species for the "Building pangenome graphs" publication. Zenodo. https://doi.org/10.5281/zenodo.7937947

[2] Marquita Ellis, Giulia Guidi, Aydın Buluç, Leonid Oliker, and Katherine Yelick. 2019. diBELLA: Distributed Long Read to Long Read Alignment. In *Proceedings of the 48th International Conference on Parallel Processing (ICPP '19)*.

[3] Erik Garrison et al. 2024. Building pangenome graphs. *Nature Methods* (2024). https://doi.org/10.1038/s41592-024-02430-3

[4] Yifan Li and Giulia Guidi. 2024. High-Performance Sorting-Based k-mer Counting in Distributed Memory with Flexible Hybrid Parallelism. In *Proceedings of the 53rd International Conference on Parallel Processing (ICPP '24)*. https://doi.org/10.1145/3673038.3673072

[5] Gene Myers, Richard Durbin, and Chenxi Zhou. 2025. FastGA: Fast Genome Alignment. *Bioinformatics Advances* (2025). https://doi.org/10.1093/bioadv/vbaf238